SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1708**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I JASMITHA R(192524295) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: JASMITHA R

Annexure A

Analytical Problem Solving

1. **Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available**.

ANSWER:

Problem Statement

You are given two water jugs:

- **Jug A = 4 gallons**
- **Jug B = 3 gallons**

There are no measuring marks on either jug.

Allowed Operations

1. Fill any jug completely from the pump.
2. Empty any jug onto the ground.
3. Pour water from one jug into the other until either:
    - the first jug becomes empty, or
    - the second jug becomes full.

Goal

Obtain **exactly 2 gallons of water in the 4-gallon jug**.

State Representation

Represent each state as:

(A, B)

Where:

- **A** = amount of water in the 4-gallon jug
- **B** = amount of water in the 3-gallon jug

Initial State:

(0, 0)

Goal State:

(2, 0) or any state where the 4-gallon jug contains exactly **2 gallons**.

Solution Steps

Step	Action	4-Gallon Jug	3-Gallon Jug
Initial	Both jugs empty	0	0
1	Fill 3-gallon jug	0	3
2	Pour into 4-gallon jug	3	0
3	Fill 3-gallon jug again	3	3
4	Pour into 4-gallon jug until full	4	2
5	Empty the 4-gallon jug	0	2
6	Pour remaining 2 gallons into 4-gallon jug	2	0

State Sequence

(0,0)
↓ Fill 3-gallon jug
(0,3)
↓ Pour into 4-gallon jug
(3,0)
↓ Fill 3-gallon jug
(3,3)
↓ Pour into 4-gallon jug
(4,2)
↓ Empty 4-gallon jug
(0,2)
↓ Pour into 4-gallon jug
(2,0) ← Goal Achieved

State Space Diagram

Start
(0,0)
|
Fill 3-gallon
|

(0,3)
|
Pour to 4-gallon
|
(3,0)
|
Fill 3-gallon
|
(3,3)
|
Pour to 4-gallon
|
(4,2)
|
Empty 4-gallon
|
(0,2)
|
Pour to 4-gallon
|
(2,0)
Goal State

Algorithm

1. Start with both jugs empty.
2. Fill the 3-gallon jug.
3. Transfer all water into the 4-gallon jug.
4. Refill the 3-gallon jug.
5. Pour water into the 4-gallon jug until it becomes full.
6. Two gallons remain in the 3-gallon jug.
7. Empty the 4-gallon jug.
8. Transfer the remaining 2 gallons into the 4-gallon jug.
9. The 4-gallon jug now contains exactly **2 gallons**.

Search Technique

The Water Jug problem is a classic **Artificial Intelligence (AI) State Space Search Problem**.

- **Initial State:** (0,0)
- **Goal State:** (2,0)

- **Operators:**
  - Fill
  - Empty
  - Pour

The problem can be solved using:

- Breadth First Search (BFS)
- Depth First Search (DFS)
- State Space Search

Conclusion

By performing a sequence of filling, pouring, and emptying operations, exactly 2 gallons of water are obtained in the 4-gallon jug without using any measuring device. This demonstrates the concept of state space search in Artificial Intelligence, where each state represents the amount of water in both jugs and operators are applied until the goal state is reached.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

i. What are the percept's for this agent?
ii. Characterize the operating environment.
iii. What are the actions the agent can take?
iv. How can one evaluate the performance of the agent?
v. What sort of agent architecture do you think is most suitable for this agent? Why?

ANSWER:

Introduction

A **Mars Rover** is an intelligent robotic agent sent to Mars to explore the planet, analyze rocks and soil, capture images, and send scientific data back to Earth. Since communication with Earth has a significant delay, the rover must operate autonomously using Artificial Intelligence.

i. What are the percepts for this agent

Percepts are the inputs received by the rover through its sensors.

The Mars Rover perceives:

- Images from cameras
- Rock and soil composition
- Surface temperature
- Atmospheric pressure
- Light intensity
- Terrain obstacles (rocks, hills, sand)
- Distance to objects

- Position and orientation (GPS-like localization using onboard navigation)
- Battery power level
- Wheel condition and motor status

ii. Characterize the Operating Environment (2 Marks)

The Mars Rover operates in the following environment:

Property	Description
Partially Observable	The rover cannot observe the entire environment at once.
Deterministic/Stochastic	Mostly stochastic because terrain and weather conditions are unpredictable.
Sequential	Current actions affect future actions.
Dynamic	Environment changes due to dust storms, lighting, and terrain conditions.
Continuous	Movement, time, and sensor readings are continuous.
Single Agent	Only one rover performs exploration independently.

iii. What are the actions the agent can take

The Mars Rover can perform the following actions:

- Move forward, backward, left, and right
- Avoid obstacles
- Capture images
- Drill rocks
- Collect soil and rock samples
- Analyze samples using onboard instruments
- Measure environmental conditions
- Send collected data to Earth
- Recharge using solar panels (if available)
- Stop or change route when obstacles are detected

iv. How can one evaluate the performance of the agent

The performance of the Mars Rover can be evaluated using:

- Successful navigation without collisions
- Accuracy of rock and soil analysis
- Number of useful scientific samples collected
- Quality of images transmitted
- Efficient use of battery power
- Ability to avoid obstacles safely
- Reliability of communication with Earth
- Completion of assigned exploration tasks within mission time

v. Suitable Agent Architecture and Why

Most Suitable Agent Architecture: Utility-Based Learning Agent

Reason:

A Utility-Based Learning Agent is best suited because it:

- Learns from previous experiences.
- Chooses the best action based on maximum utility.
- Adapts to unknown and changing environments.
- Makes autonomous decisions when communication with Earth is delayed.
- Optimizes energy consumption and mission objectives.
- Improves navigation and sample collection over time.

PEAS Representation of Mars Rover

Component	Description
Performance Measure	Safe navigation, accurate analysis, successful sample collection, battery efficiency
Environment	Mars surface, rocks, sand, dust storms, craters, atmosphere
Actuators	Wheels, robotic arm, drill, camera, communication antenna
Sensors	Cameras, spectrometers, temperature sensors, proximity sensors, gyroscope, battery sensors

Conclusion

The **Mars Rover** is an intelligent autonomous AI agent that explores Mars by sensiits surroundings, navigating safely, collecting and analyzing samples, and transmitting scientific information to Earth. A **Utility-Based Learning Agent** is the most suitable architecture because it enables the rover to make intelligent decisions, adapt to changing conditions, and maximize mission success while operating independently.

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

Problem Statement

The 8-Queens Problem is a classic Artificial Intelligence search problem. The objective is to place 8 queens on an 8 × 8 chessboard such that no two queens attack each other.

A queen can attack another queen if they are in the:

- Same row
- Same column
- Same diagonal

The given configuration is not a solution because the queen in the first column and the queen in the last column are on the same diagonal.

Problem Formulation

1. Initial State

- An empty 8 × 8 chessboard.
- No queens are placed.

State:

0 Queens on the board

2. State Space

A state represents a partial arrangement of queens on the chessboard.

Example:

Q . . . . . .
. . Q . . . . .
. . . . Q . . .
. Q . . . . . .
. . . . . Q . .
. . . Q . . . .
. . . . . Q .
. . . . . Q

Each new state is generated by placing one queen in a safe position.

3. Operators (Actions)

The possible actions are:

- Place a queen in a safe square.
- Move a queen (in local search methods).
- Remove and replace a queen if necessary.

4. Goal State

The goal is reached when:

- All **8 queens** are placed.
- No two queens attack each other.

That means:

- No two queens share the same row.
- No two queens share the same column.
- No two queens share the same diagonal.

Search Space

The search space contains all possible arrangements of queens.

- Without restrictions:

[
8^8=16,777,216
]
possible arrangements.

By applying constraints (one queen per row and column), the search space is greatly reduced.

Search Strategies

1. Breadth First Search (BFS)

- Explores all possible states level by level.
- Guarantees finding a solution.
- Requires large memory.

Advantages

- Complete
- Finds shortest path

Disadvantages

- High memory usage
- Slow for large search spaces

2. Depth First Search (DFS)

- Places queens row by row.
- Backtracks when a conflict occurs.

Advantages

- Uses less memory
- Easy to implement

Disadvantages

- May explore unnecessary paths
- Not always optimal

3. Backtracking Search (Most Common)

Backtracking is the preferred strategy for solving the 8-Queens problem.

Algorithm:

1. Place a queen in the first row.
2. Move to the next row.
3. If the position is safe, place another queen.
4. If no safe position exists, backtrack.
5. Continue until all eight queens are placed.

State Space Representation

Start

|

Empty Board

|

Place Queen in Row 1

|

Place Queen in Row 2
|
Conflict?
/ \
Yes No
| |
Backtrack Continue
|
Place Remaining Queens
|
Goal State

Performance Analysis

Search Strategy	Memory	Time	Complete
BFS	High	High	Yes
DFS	Low	Moderate	No
Backtracking	Low	Efficient	Yes

Conclusion

The 8-Queens Problem is a classic Constraint Satisfaction Problem (CSP) in Artificial Intelligence. It is solved by representing each board configuration as a state and exploring the search space using strategies such as BFS, DFS, and Backtracking. Among these, Backtracking Search is the most efficient because it checks constraints at each step and prunes invalid solutions early. It successfully places all 8 queens on the chessboard so that no two queens attack each other.

4. **A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.**

ANSWER:

Introduction

An OLA Cab Booking System is an example of a Goal-Based Problem Solving Agent. The agent helps the customer reach the destination by selecting the most suitable cab based on preferences such as cost, comfort, availability, and travel time.

Type of Problem-Solving Agent

Goal-Based Agent

A Goal-Based Agent is the most suitable because:

- It has a specific goal: reach the destination.

- It evaluates different cab options (Mini, Micro, Sedan, Prime, Shared, etc.).
- It selects the cab that best satisfies the user's preferences.
- It plans and chooses the best action to achieve the goal.

Problem Formulation

Initial State

- Customer opens the OLA application.
- Current location and destination are entered.

Actions

- Detect current location.
- Enter destination.
- Display available cab types.
- Compare fare and estimated arrival time.
- Select a preferred cab.
- Confirm booking.
- Driver accepts the request.
- Start the trip.
- Reach the destination.

Goal State

- Customer safely reaches the desired destination.

PEAS Representation

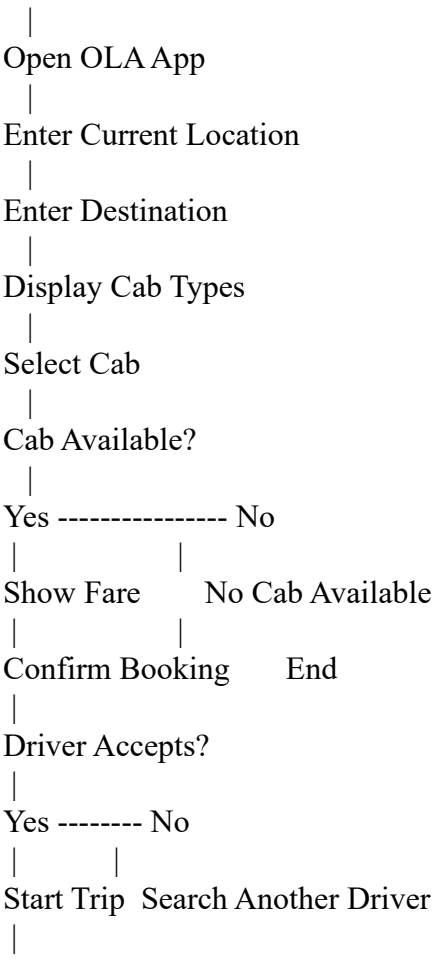
Component	Description
Performance Measure	Minimum fare, shortest travel time, customer satisfaction, safe journey
Environment	Roads, traffic, drivers, customers, GPS, weather
Actuators	Mobile app display, booking request, payment system, notifications
Sensors	GPS, internet, customer location, destination, traffic updates

Pseudocode

```
BEGIN

Input Current_Location
Input Destination

Display Available_Cab_Types
(Mini, Micro, Sedan, Prime, Shared)

Customer selects Cab_Type

Check Cab Availability

IF Cab is Available THEN
    Calculate Fare
    Display Estimated Time
```

```
    Confirm Booking

    Assign Driver

    IF Driver Accepts THEN
        Start Trip
        Navigate to Destination
        Complete Ride
        Process Payment
        Display "Trip Completed Successfully"
    ELSE
        Display "Searching for Another Driver"
    ENDIF

ELSE
    Display "No Cabs Available"
ENDIF

END
```

Flowchart



Reach Destination
|
Payment
|
Trip Completed
|
End

Working of the Agent

1. The customer enters the pickup and destination locations.
2. The system detects the current location using GPS.
3. Available cab types are displayed.
4. The customer selects a preferred cab.
5. The system checks availability.
6. A nearby driver is assigned.
7. The driver accepts the booking.
8. The customer travels to the destination.
9. Payment is completed and the trip ends.

Advantages

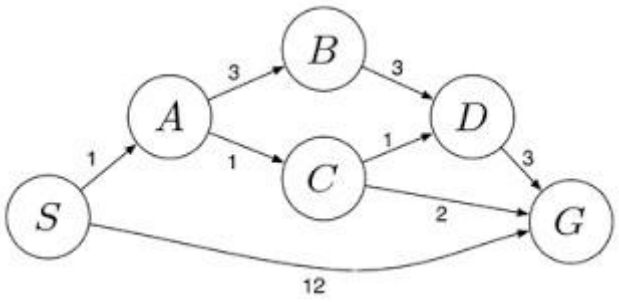
- Finds the best available cab.
- Reduces waiting time.
- Optimizes travel cost.
- Provides safe and convenient transportation.
- Gives real-time tracking and navigation.

Conclusion

The OLA Cab Booking Application is best modeled as a Goal-Based Problem Solving Agent because it works toward a specific goal—transporting the customer from the source to the destination. The agent gathers information, evaluates available cab options, selects the most suitable one based on the user's preferences, and successfully completes the journey.

5. **A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.**

The delivery network is represented as a weighted graph:



ANSWER:

Problem Statement

A logistics company wants to transport packages from the starting warehouse S to the destination warehouse G with the minimum travel cost. The network is represented as a weighted graph, where each edge has an associated travel cost. The **Uniform Cost Search (UCS)** algorithm is used to find the least-cost path.

Given Graph

Edges and Costs:

- S → A = **1**
- S → G = **12**
- A → B = **3**
- A → C = **1**
- B → D = **3**
- C → D = **1**
- C → G = **2**
- D → G = **3**

Uniform Cost Search (UCS)

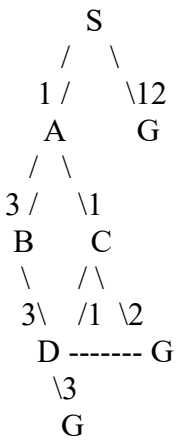
Uniform Cost Search expands the node with the lowest cumulative path cost first. It guarantees the optimal (least-cost) path when all edge costs are non-negative.

Step-by-Step UCS Execution

Step	Node Expanded	Path Cost	Frontier After Expansion
1	S	0	A(1), G(12)
2	A	1	C(2), B(4), G(12)
3	C	2	D(3), G(4), B(4)

4	D	3	G(4), B(4), G(6)
5	G	4	Goal Reached

Search Tree



Possible Paths

Path	**Total Cost**
S → G	12
S → A → B → D → G	1 + 3 + 3 + 3 = **10**
S → A → C → D → G	1 + 1 + 1 + 3 = **6**
S → A → C → G	**1 + 1 + 2 = 4**

Least-Cost Path Optimal Path:

S → A → C → G

Total Cost:

1 + 1 + 2 = 4

Algorithm (UCS Pseudocode)

BEGIN

1. Insert Start node (S) into the priority queue with cost 0.

2. Repeat until the queue is empty:

a. Remove the node with the smallest path cost.

b. If the node is the Goal (G), stop and return the path.

c. Expand the node.

d. Add all child nodes with their cumulative costs to the priority queue.

3. If the queue becomes empty, return Failure.

END

Advantages of UCS

- Finds the optimal (minimum-cost) path.
- Expands nodes in order of increasing path cost.
- Complete when all costs are positive.
- Suitable for routing, logistics, and navigation problems.

Conclusion

Using the Uniform Cost Search (UCS) algorithm, the least-cost route from the starting warehouse **S** to the destination warehouse **G** is:

S → A → C → G

with a minimum total cost of 4. UCS guarantees this optimal solution by always expanding the node with the lowest cumulative path cost first.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Problem Understanding	20%	Clearly interprets problem, identifies all parameters and	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
		requirements accurately			